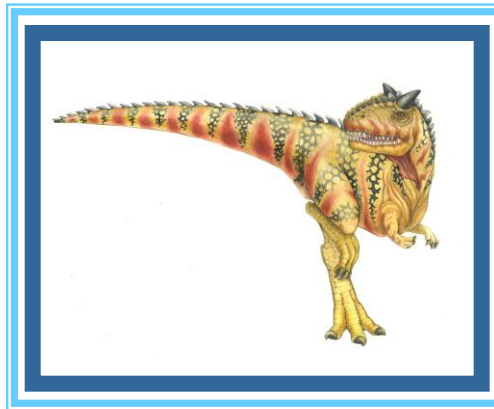
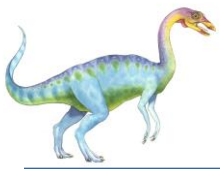


Chapter 8: Main Memory





Chapter 8: Memory Management

- ❑ Background
- ❑ Swapping
- ❑ Contiguous Memory Allocation
- ❑ Segmentation
- ❑ Paging
- ❑ Structure of the Page Table





Background

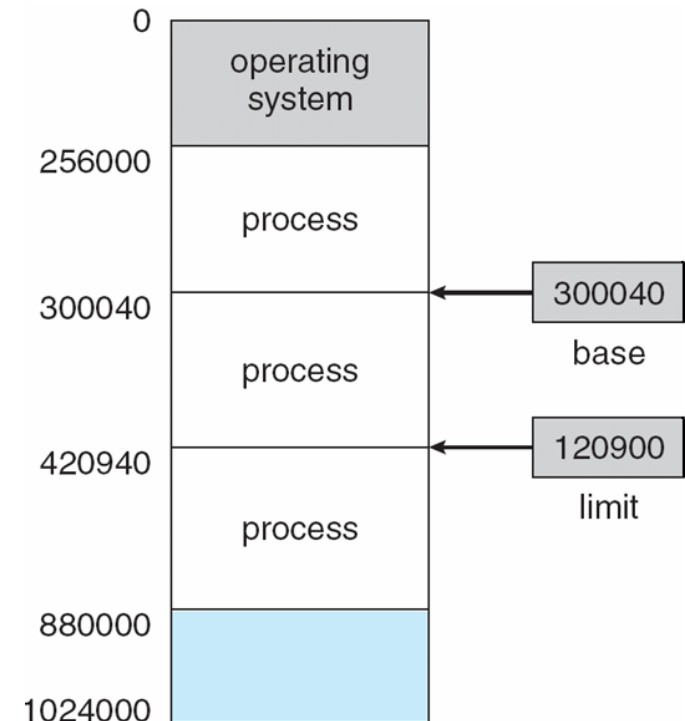
- ❑ Program must be brought (from disk) into memory and placed within a process for it to be run
- ❑ Main memory and registers are only storage CPU can access directly
- ❑ Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- ❑ Register access in one CPU clock (or less)
- ❑ Main memory can take many cycles, causing a **stall**
- ❑ **Cache** sits between main memory and CPU registers
- ❑ Protection of memory required to ensure correct operation
- ❑ must ensure correct operation to protect the operating system from access by user processes and, in addition, to protect user processes from one another.
- ❑ This protection must be provided by the hardware





Base and Limit Registers

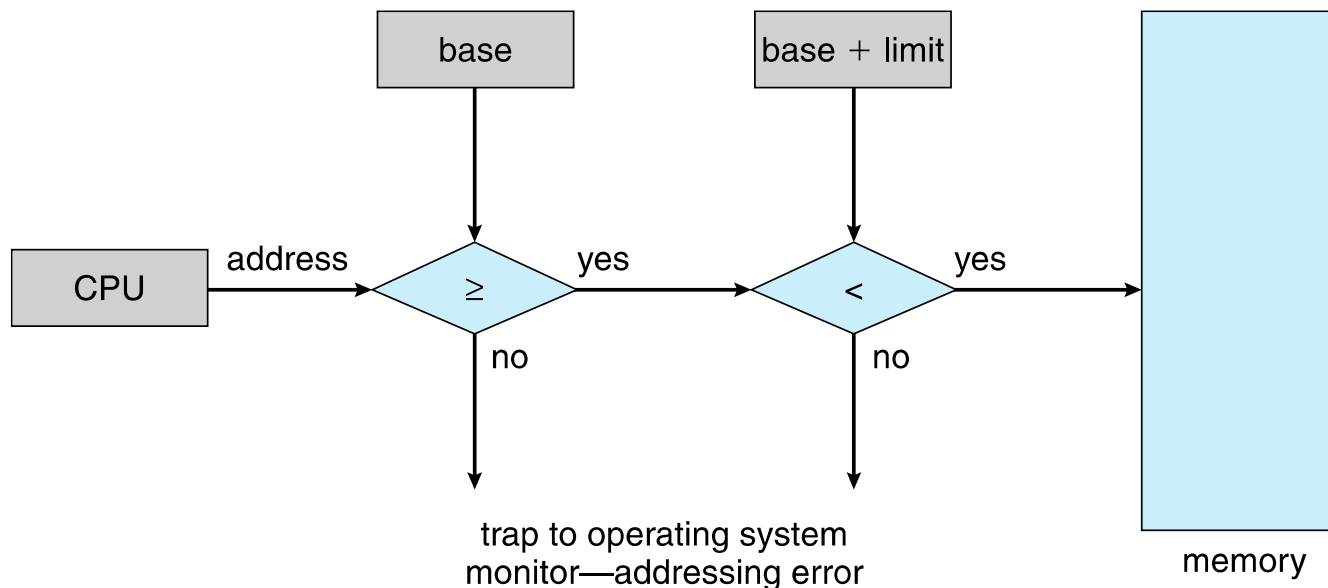
- ❑ need to make sure that each process has a separate memory space.
 - ❑ To do this, we need the ability to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.
 - ❑ Provided by using two registers, usually a **base** and **limit** registers
 - ❑ The base register holds the smallest legal physical memory address; the limit register specifies the size of the range.
 - ❑ For example, if the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939 (inclusive).
 - ❑ A pair of base and limit registers define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user





Hardware Address Protection

- Protection of memory space is accomplished by having the CPU hardware
- compare *every* address generated in user mode with the registers.
- Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error
- This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users.





Hardware Address Protection Contd..

- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.
- Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.
- This scheme allows the operating system to change the value of the registers but prevents user programs from changing the registers' contents.





Address Binding

- Programs on disk, ready to be brought into memory to execute form an **input queue**
 - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
- Further, addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic
 - Compiled code addresses **bind** to relocatable addresses
 - ▶ i.e. “14 bytes from beginning of this module”
 - Linker or loader will bind relocatable addresses to absolute addresses
 - ▶ i.e. 74014
 - Each binding maps one address space to another





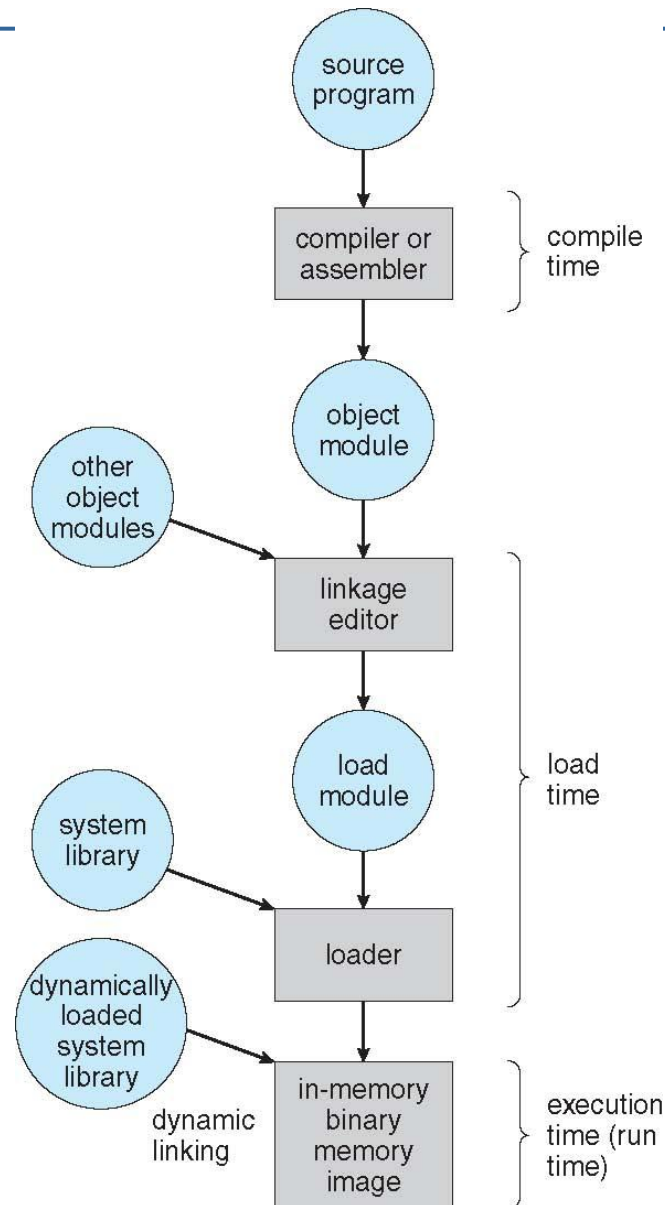
Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - ▶ binding is delayed until load time.
 - ▶ If the starting address changes, we need only reload the user code to incorporate this changed value.
 - **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another
 - ▶ Need hardware support for address maps





Multistep Processing of a User Program





Logical vs. Physical Address Space

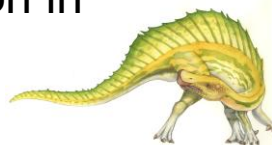
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address**
 - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program





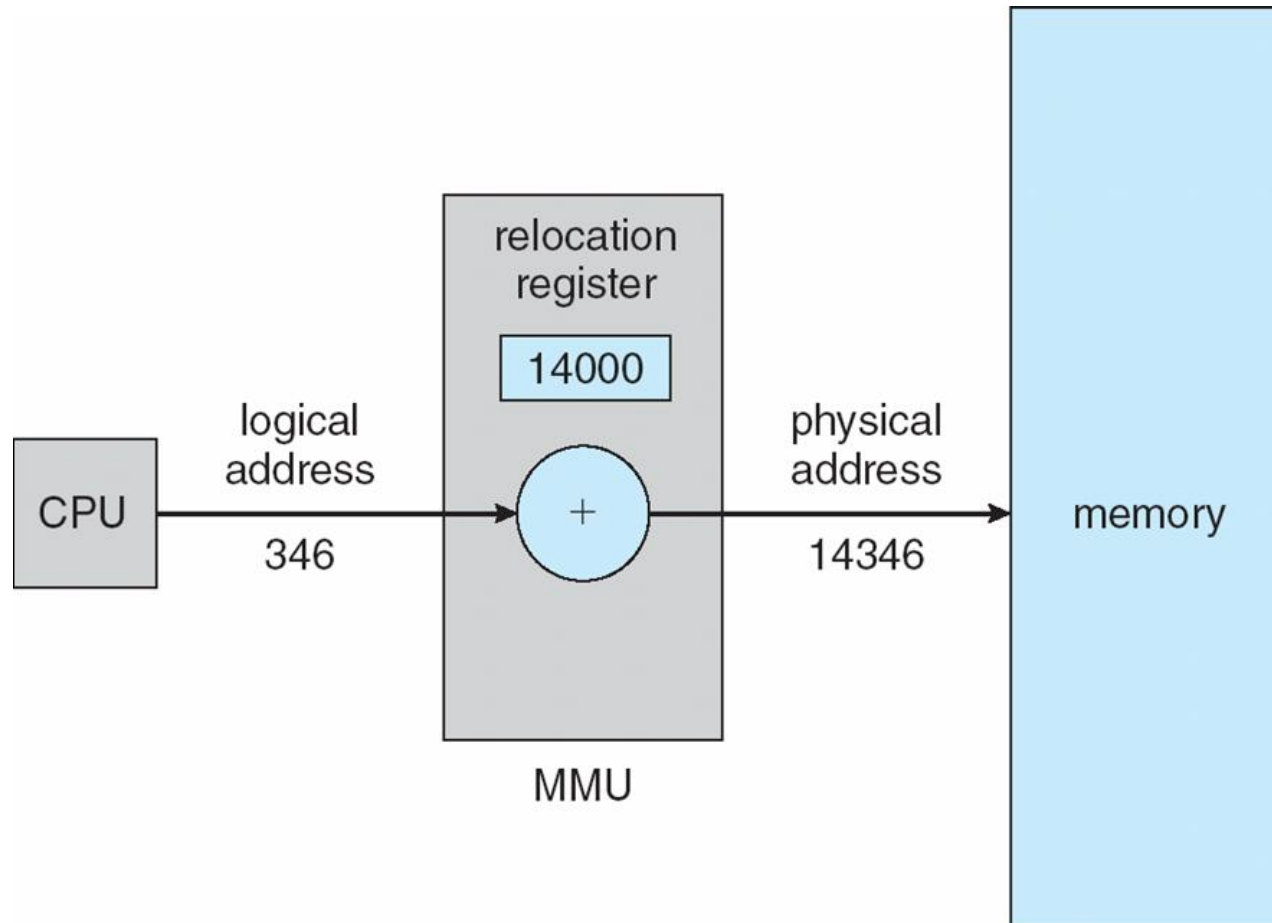
Memory-Management Unit (MMU)

- ❑ Hardware device that at run time maps virtual to physical address
- ❑ Many methods possible
- ❑ To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
 - ❑ Base register now called **relocation register**
- ❑ For example,
- ❑ if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.
- ❑ The MS-DOS operating system running on the Intel 80x86 family of processors used four relocation registers when loading and running processes.
- ❑ The user program deals with *logical* addresses; it never sees the *real* physical addresses
 - ❑ Execution-time binding occurs when reference is made to location in memory
 - ❑ Logical address bound to physical addresses





Dynamic relocation using a relocation register





Memory-Management Unit (MMU) Contd..

- have two different types of addresses: logical addresses (in the range 0 to max) and physical addresses (in the range $R + 0$ to $R + max$ for a base value R).
- The user program generates only logical addresses and thinks that the process runs in locations 0 to max .
- However, these logical addresses must be mapped to physical addresses before they are used





Dynamic loading

- ❑ necessary for the entire program and all data of a process to be in physical memory for the process to execute.
- ❑ The size of a process has thus been limited to the size of physical memory.
- ❑ To obtain better memory-space utilization, we can use dynamic loading.
- ❑ Routine is not loaded until it is called : unused routine is never loaded
- ❑ All routines kept on disk in relocatable load format
- ❑ The main program is loaded into memory and is executed.
- ❑ When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded. If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.
- ❑ Then control is passed to the newly loaded routine.
- ❑ Useful when large amounts of code are needed to handle infrequently occurring cases
- ❑ No special support from the operating system is required
 - ❑ Implemented through program design
 - ❑ OS can help by providing libraries to implement dynamic loading





Dynamic Linking

- ❑ **Static linking** – system libraries and program code combined by the loader into the binary program image
- ❑ Dynamic linking – linking postponed until execution time
- ❑ Without this facility, each program on a system must include a copy of its language library (or at least the routines referenced by the program) in the executable image.
 - ❑ This requirement wastes both disk space and mainmemory.
- ❑ Small piece of code, **stub**, used to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.
- ❑ When the stub is executed, it checks to see whether the needed routine is already in memory. If it is not, the program loads the routine into memory.
- ❑ Stub replaces itself with the address of the routine, and executes the routine
- ❑ Operating system checks if routine is in processes' memory address
 - ❑ If not in address space, add to address space
- ❑ Dynamic linking is particularly useful for libraries
- ❑ System also known as **shared libraries**





Dynamic Linking Contd...

- ❑ This feature can be extended to library updates (such as bug fixes).
- ❑ A library may be replaced by a new version, and all programs that reference the library will automatically use the new version.
- ❑ Without dynamic linking, all such programs would need to be relinked to gain access to the new library.
- ❑ So that programs will not accidentally execute new, incompatible versions of libraries, version information is included in both the program and the library
- ❑ More than one version of a library may be loaded into memory, and each program uses its version information to decide which copy of the library to use.
- ❑ Versions with minor changes retain the same version number, whereas versions with major changes increment the number.
- ❑ Thus, only programs that are compiled with the new library version are affected by any incompatible changes incorporated
- ❑ Other programs linked before the new library was installed will continue
- ❑ using the older library.
- ❑ This System also known as **shared libraries**
- ❑ **Needs support from the operating system**





Swapping

- ❑ A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - ❑ Total physical memory space of processes can exceed physical memory
- ❑ **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- ❑ **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images.
- ❑ System maintains a **ready queue** of ready-to-run processes which have memory images on disk
- ❑ Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.
- ❑ The dispatcher checks to see whether the next process in the queue is in memory.
- ❑ If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process. It then reloads registers and transfers control to the selected process.
- ❑ Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped





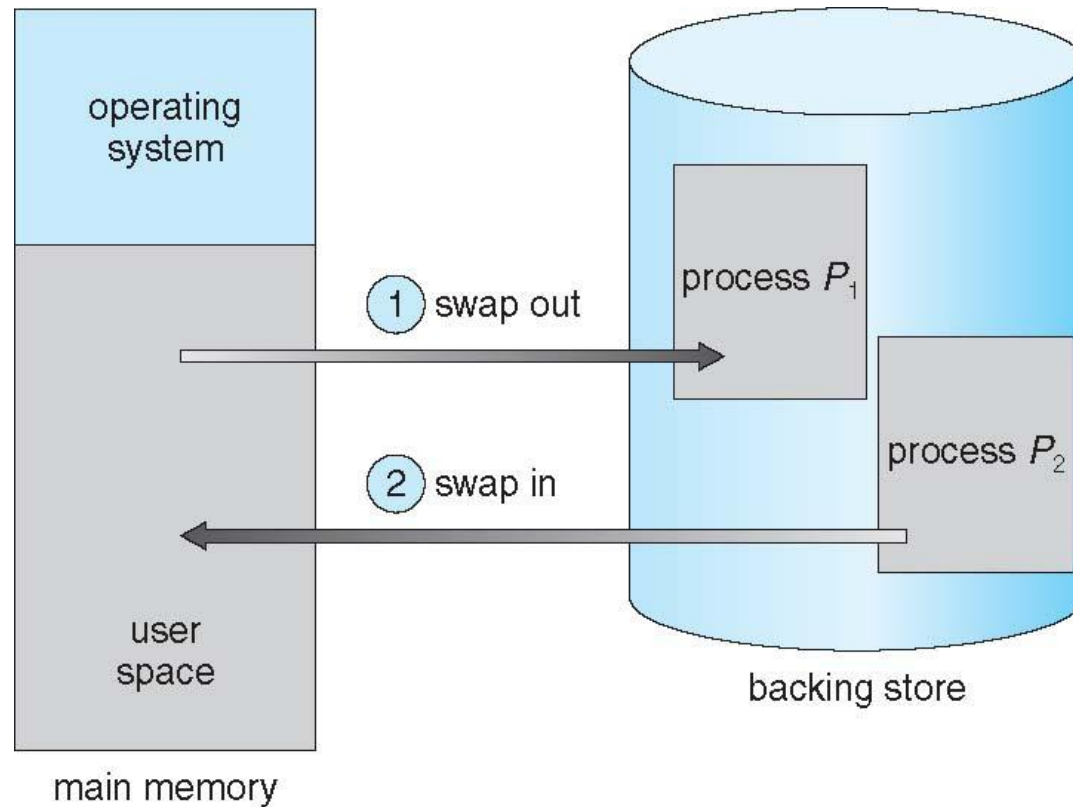
Swapping (Cont.)

- ❑ Does the swapped out process need to swap back in to same physical addresses?
- ❑ a process that is swapped out will be swapped back into the same memory space it occupied previously.
- ❑ Depends on address binding method
- ❑ If binding is done at assembly or load time, then the process cannot be easily moved to a different location.
- ❑ If execution-time binding is being used, however, then a process can be swapped into a different memory space, because the physical addresses are computed during execution time.





Schematic View of Swapping





Context Switch Time including Swapping

- ❑ If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- ❑ Context switch time can then be very high
- ❑ 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - ❑ Swap out time of 2000 ms
 - ❑ Plus swap in of same sized process
 - ❑ Total context switch swapping component time of 4000ms (4 seconds)
- ❑ Can reduce if reduce size of memory swapped – by knowing how much memory really being used
 - ❑ System calls to inform OS of memory use via `request_memory()` and `release_memory()`





Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
- Pending I/O – can't swap out as I/O would occur to wrong process-
two main solutions to this problem:
 - never swap a process with pending I/O
 - execute I/O operations only into operating-system buffers.
 - ▶ Transfers between operating-system buffers and process memory then occur only when the process is swapped in.
- Standard swapping not used in modern operating systems
 - But modified version common
 - ▶ Swap only when free memory extremely low





Contiguous Allocation

- ❑ Main memory must support both OS and user processes
- ❑ Limited resource, must allocate efficiently
- ❑ Contiguous allocation is one early method
- ❑ Main memory usually into two **partitions**:
 - ❑ Resident operating system, usually held in low memory with interrupt vector
 - ❑ User processes then held in high memory
 - ❑ Each process contained in single contiguous section of memory





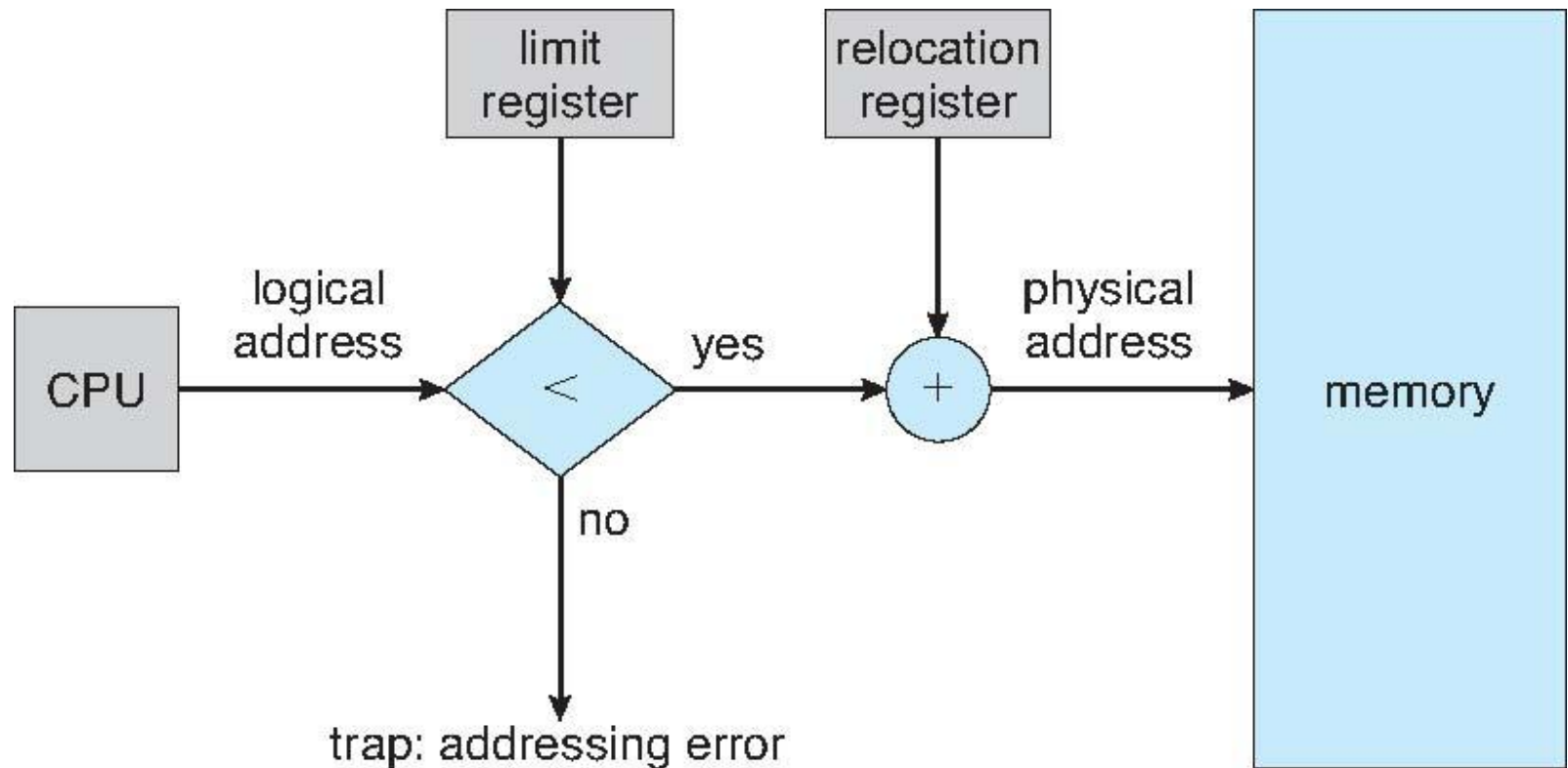
Contiguous Allocation (Cont.)

- ❑ Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - ❑ Base register contains value of smallest physical address
 - ❑ Limit register contains range of logical addresses – each logical address must be less than the limit register
 - ❑ MMU maps logical address *dynamically*
 - ❑ When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch.
 - ❑ Because every address generated by a CPU is checked against these registers, we can protect both the operating system and other users'
 - ❑ programs and data from being modified by this running process
 - ❑ Can then allow actions such as kernel code being **transient** and kernel changing size





Hardware Support for Relocation and Limit Registers





Memory allocation

- Simplest method for allocating memory is to divide memory into several fixed-sized **partitions**.
- Each partition may contain exactly one process.
- Degree of multiprogramming limited by number of partitions
- Multiple-partition allocation - when a partition is free, a process is selected from the input queue and is loaded into the free partition.
- When the process terminates, the partition becomes available for another process
- Originally used by the IBM OS/360 operating system (called MFT); it is no longer in use





Memory allocation- Variable-partition

- ❑ **Variable-partition** sizes for efficiency (sized to a given process' needs)
- ❑ operating system keeps a table indicating which parts of memory are available and which are occupied.
- ❑ Initially, all memory is available for user processes and is considered one large block of available memory, **a hole**
- ❑ As processes enter the system, they are put into an input queue.
 - ❑ The OS takes into account the memory requirements of each process and the amount of available memory space
- ❑ When a process is allocated space, it is loaded into memory, and it can then compete for CPU time.
- ❑ When a process terminates, it releases its memory.
- ❑ At any given time, then, we have a list of available block sizes and an input queue. The OS can order the input queue according to a scheduling algorithm.





Variable-partition Contd...

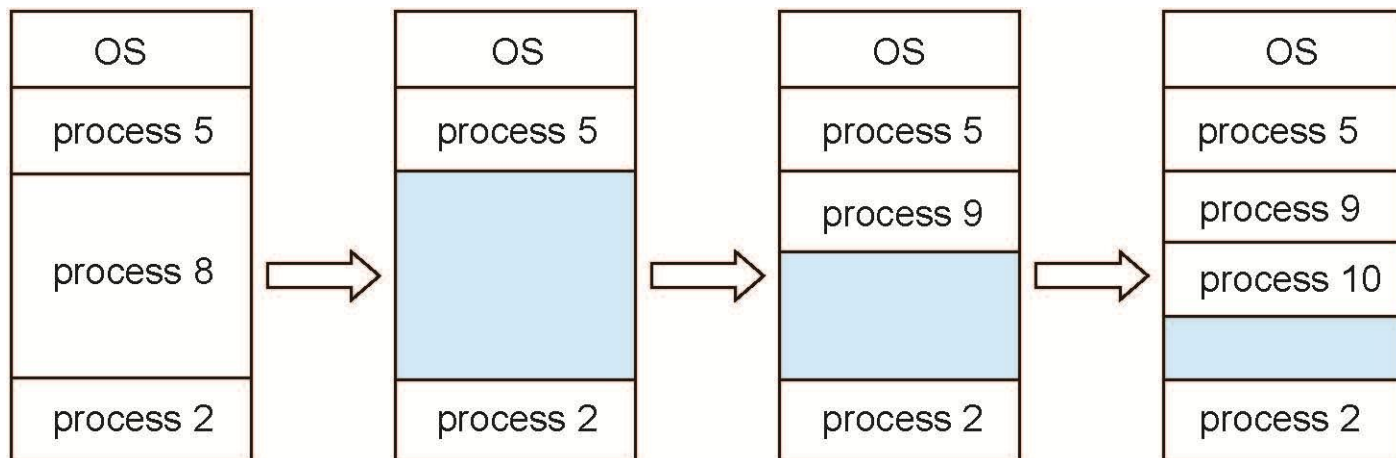
- ❑ Memory is allocated to processes until, finally, the memory requirements of the next process cannot be satisfied—that is, no available block of memory (or hole) is large enough to hold that process.
- ❑ The operating system can then wait until a large enough block is available, or it can skip down the input queue to see whether the smaller memory requirements of some other process can be met.

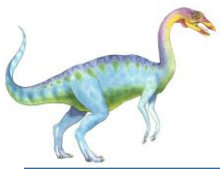




Variable-partition Contd...

- ❑ **Hole** – block of available memory; holes of various size are scattered throughout memory
- ❑ When a process arrives, it is allocated memory from a hole large enough to accommodate it
- ❑ Process exiting frees its partition, adjacent free partitions combined
- ❑ Operating system maintains information about:
a) allocated partitions b) free partitions (hole)





Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes?

- **First-fit**: Allocate the **first** hole that is big enough
- **Best-fit**: Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
- **Worst-fit**: Allocate the **largest** hole; must also search entire list
 - Produces the largest leftover hole
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization
- Neither first fit nor best fit is clearly better than the other in terms of storage utilization, but first fit is generally faster.





Fragmentation- External fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
 - Could have a block of free (or wasted) memory between every two processes.
 - If all these small pieces of memory were in one big free block instead, we might be able to run several more processes.
 - Depending on the total amount of memory storage and the average process size, external fragmentation may be a minor or a major problem
 - First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation
 - $1/3$ may be unusable -> **50-percent rule**

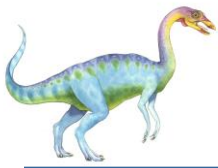




Fragmentation- internal fragmentation

- Memory fragmentation can be internal as well as external.
- Consider a multiple-partition allocation scheme with a hole of 18,464 bytes.
- Suppose that the next process requests 18,462 bytes.
- If we allocate exactly the requested block, we are left with a hole of 2 bytes. The overhead to keep track of this hole will be substantially larger than the hole itself.
- The general approach -break the physical memory into fixed-sized blocks and allocate memory in units based on block size.
- With this approach, the memory allocated to a process may be slightly larger than the requested memory.
 - The difference between these two numbers is **internal fragmentation**—unused memory that is internal to a partition.





Fragmentation (Cont.)

- ❑ Reduce external fragmentation by **compaction**
 - ❑ Shuffle memory contents to place all free memory together in one large block
 - ❑ Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - ▶ requires only moving the program and data and then changing the base register to reflect the new base address.
- ❑ simplest compaction algorithm is to move all processes toward one end of memory; all holes move in the other direction, producing one large hole of available memory.
- ❑ This scheme can be expensive
- ❑ possible solution to external-fragmentation problem - permit the logical address space of the processes to be noncontiguous - allowing a process to be allocated physical memory wherever such memory is available.
- ❑ Two techniques - paging and segmentation





Exercise

Given six memory partitions of 100 MB, 170 MB, 40 MB, 205 MB, 300 MB, and 185 MB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 200 MB, 15 MB, 185 MB, 75 MB, 175 MB, and 80 MB (in order)? Indicate which—if any—requests cannot be satisfied. Comment on how efficiently each of the algorithms manages memory





Approach: Allocate the first partition that is large enough.

100 MB, 170 MB, 40 MB, 205 MB, 300 MB, and 185 MB (in order),

Process	Size
P1	200
P2	15
P3	185
P4	75
P5	175
P6	80





□ First-fit algorithm

- P1 allocated to F4 with remaining space of $205 - 200 = 5\text{MB}$.
- P2 allocated to F1 with remaining space of $100 - 15 = 85\text{MB}$
- P3 allocated to F5 with remaining space of $300 - 185 = 115\text{MB}$.
- P4 allocated to F1 with remaining space of $85 - 75 = 10\text{MB}$
- P5 allocated to F6 with remaining space of $185 - 175 = 10\text{MB}$
- P6 allocated to F2 with remaining space of $170 - 80 = 90\text{MB}$

.





□ **Best-fit algorithm**

- P1 allocated to F4 with remaining space of $205 - 200 = 5\text{MB}$.
- P2 allocated to F3 with remaining space of $40 - 15 = 25\text{MB}$
- P3 was allocated to F6 but is entirely occupied, so no remaining space will be there
- P4 allocated to F1 with remaining space of $100 - 75 = 25\text{MB}$
- P5 allocated to F5 with remaining space of $300 - 175 = 125\text{MB}$
- P6 allocated to F5 with remaining space of $125 - 80 = 45\text{MB}$

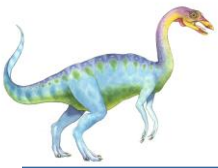




❑ **Worst-fit Algorithm**

- ❑ P1 allocated to F5 with remaining space of $300 - 200 = 100\text{MB}$.
- ❑ P2 allocated to F4 with remaining space of $205 - 15 = 190\text{MB}$
- ❑ P3 allocated to F4 with the remaining space of $190 - 185 = 5\text{MB}$.
- ❑ P4 allocated to F6 with remaining space of $185 - 75 = 110\text{MB}$.
- ❑ P5 does not contain any memory available spaces.
- ❑ P6 allocated to F2 with remaining space of $170 - 80 = 90\text{MB}$.
- ❑ According to this, the best-fit algorithm located and manages the memory very well whereas the worst-fit algorithm does not allocate the memory properly.





- Scenario:
- **Memory Blocks (in KB):** 100, 500, 200, 300, 600
- **Processes (in KB):** 212, 417, 112, 426





- 1. First Fit
- **Method:** The process is allocated to the first free block that is large enough to hold it.
- **Placement:**
 - **212 KB:** Placed in the 500 KB block – 288 KB Remaining
 - **417 KB:** Placed in the 600 KB block
 - **112 KB:** Placed in the 288 KB block remaining from 500K.
 - **426 KB:** Cannot be placed, as no remaining blocks are large enough.
- **Result:** 426 KB process cannot be accommodated.





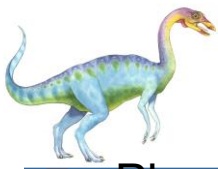
- 2. Best Fit
- **Memory Blocks (in KB):** 100, 500, 200, 300, 600
- **Processes (in KB):** 212, 417, 112, 426
- **Method:** The process is allocated to the smallest free block that can accommodate it, minimizing internal fragmentation.
- **Placement:**
 - **212 KB:** Placed in the 300 KB block (smallest block that fits).
 - **417 KB:** Placed in the 500 KB block (smallest block that fits).
 - **112 KB:** Placed in the 200 KB block (smallest block that fits).
 - **426 KB:** Placed in the 600 KB block (smallest block that fits).
- **Result:** All processes are placed efficiently.





- 3. Worst Fit
- **Memory Blocks (in KB):** 100, 500, 200, 300, 600
- **Processes (in KB):** 212, 417, 112, 426
- **Method:** The process is allocated to the largest free block available, aiming to leave large remaining fragments that can be used for larger future processes.
- **Placement:**
 - **212 KB:** Placed in the 600 KB block-388KB Remaining
 - **417 KB:** Placed in the 500 KB block -83KB Remaining
 - **112 KB:** Placed in the 388 KB block Remaining from 600KB
 - **426 KB:** Cannot be placed, as no remaining blocks are large enough.
- **Result:** 426 KB process cannot be accommodated.





Paging

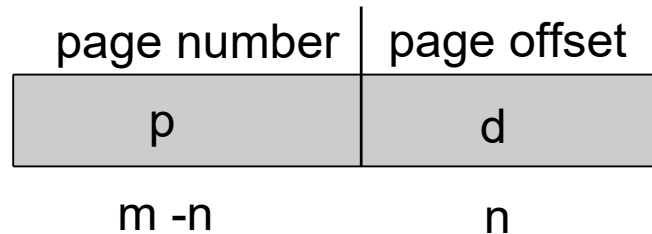
- ❑ Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - ❑ Avoids external fragmentation
 - ❑ Avoids problem of varying sized memory chunks ; fitting memory chunks of varying sizes onto the backing store
- ❑ Divide physical memory into fixed-sized blocks called **frames**
 - ❑ Size is power of 2, between 512 bytes and 16 Mbytes
- ❑ If the size of the logical address space is 2^m and a page size is 2^n addressing units (bytes or words), then the high-order $m - n$ bits of a logical address designate the page number and the n low-order bits designate the page offset.
- ❑ Divide logical memory into blocks of same size called **pages**
- ❑ Keep track of all free frames
- ❑ To run a program of size **N** pages, need to find **N** free frames and load program
- ❑ Set up a **page table** to translate logical to physical addresses
- ❑ Backing store likewise split into pages
- ❑ Still have Internal fragmentation





Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit

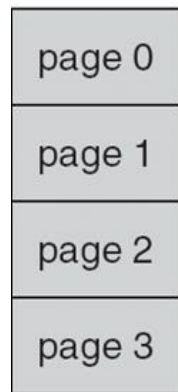


- For given logical address space 2^m and page size 2^n





Paging Model of Logical and Physical Memory

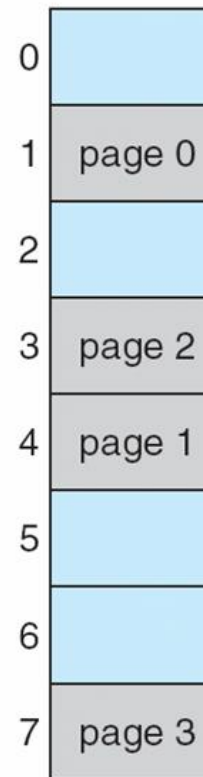


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

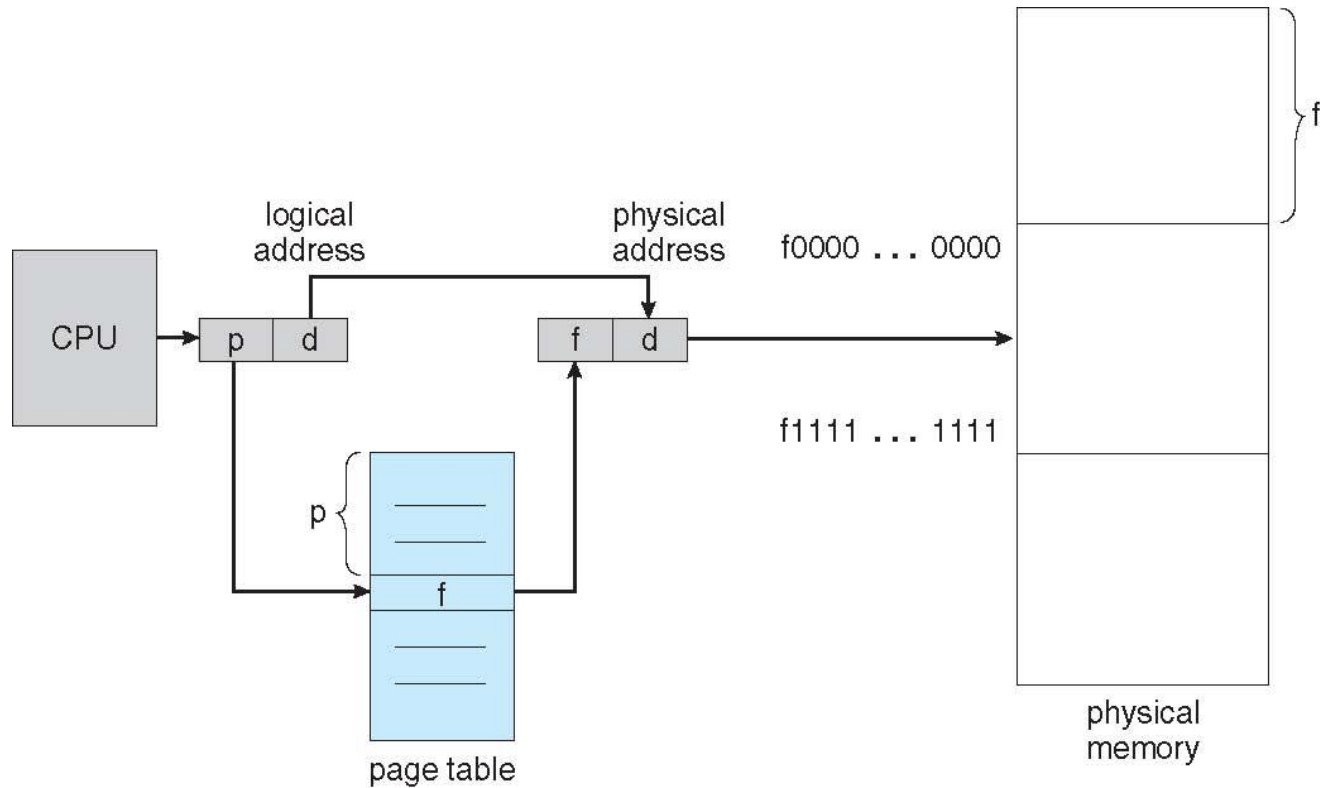


physical
memory





Paging Hardware





Paging Example

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

$n=2$ and $m=4$
32-byte memory
and 4-byte
pages





Paging (Cont.)

- Calculating internal fragmentation
 - Page size = 2,048 bytes
 - Process size = 72,766 bytes
 - 35 pages + 1,086 bytes
 - Internal fragmentation of $2,048 - 1,086 = 962$ bytes
 - Worst case fragmentation = 1 frame – 1 byte
 - On average fragmentation = $1 / 2$ frame size
 - So small frame sizes desirable?
 - But each page table entry takes memory to track
 - Page sizes growing over time
 - ▶ Solaris supports two page sizes – 8 KB and 4 MB
- Process view and physical memory now very different
- By implementation process can only access its own memory





Paging (Cont.)

- When a process arrives in the system to be executed, its size, expressed in pages, is examined.
- Each page of the process needs one frame.
- If the process requires n pages, at least n frames must be available in memory.
- If n frames are available, they are allocated to this arriving process.
- The first page of the process is loaded into one of the allocated frames, and the frame number is put in the page table for this process.
- The next page is loaded into another frame, its frame number is put into the page table, and so on





Paging (Cont.)

- ❑ The user program views memory as one single space containing only this one program.
- ❑ User program is scattered throughout physical memory, which also holds other programs.
- ❑ The difference between the user's view of memory and the actual physical memory is reconciled by the address-translation hardware.
- ❑ The logical addresses are translated into physical addresses.
- ❑ This mapping is hidden from the user and is controlled by the operating system.
- ❑ user process is unable to access memory it does not own.
- ❑ It has no way of addressing memory outside of its page table, and the table includes only those pages that the process owns.





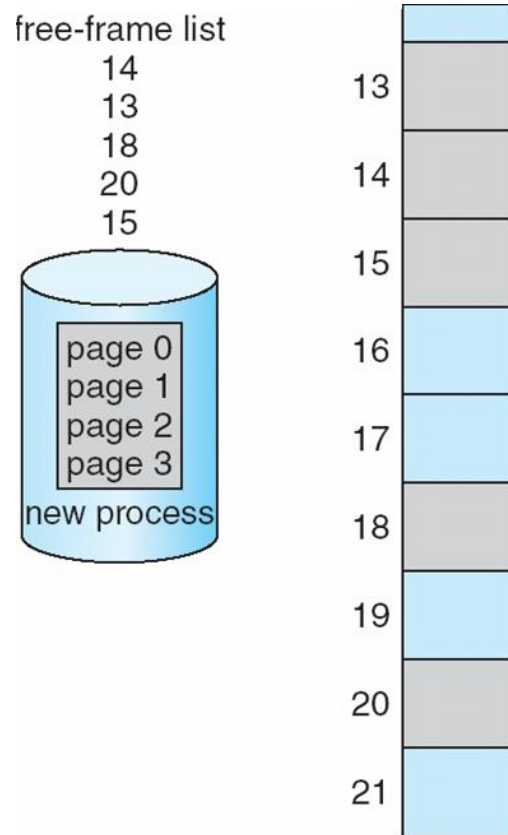
Paging (Cont.)

- Since the operating system is managing physical memory, it must be aware of the allocation details of physical memory—which frames are allocated, which frames are available, how many total frames there are, and so on.
- This information is generally kept in a data structure called a **frame table**.
- The frame table has one entry for each physical page frame, indicating whether the latter is free or allocated and, if it is allocated, to which page of which process or processes.
- The operating system maintains a copy of the page table for each process,.
 - used to translate logical addresses to physical addresses whenever the operating system must map a logical address to a physical address manually.



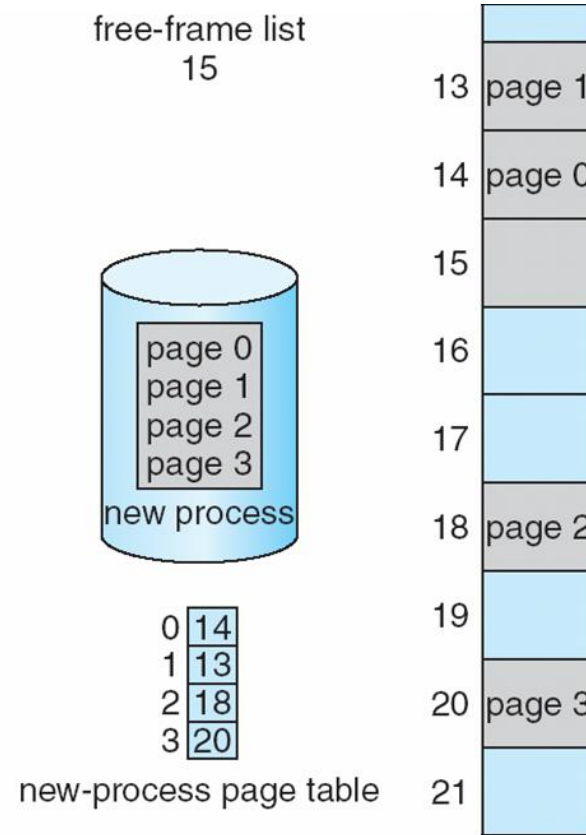


Free Frames



(a)

Before allocation



(b)

After allocation

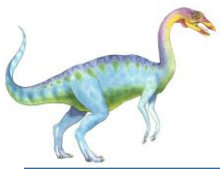




Hardware Support

- Most allocate a page table for each process.
- A pointer to the page table is stored with the other register values (like the instruction counter) in the process control block.
- When the dispatcher is told to start a process, it must reload the user registers and define the correct hardware page-table values from the stored user page table.





Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
 - Changing page tables requires changing this register
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**
- Each entry in the TLB consists of two parts: a key (or tag) and a value.
- When the associative memory is presented with an item, the item is compared with all keys simultaneously.
- If the item is found, the corresponding value field is returned.
- The search is fast; the hardware, however, is expensive.
- Typically, the number of entries in a TLB is small, often numbering between 64 and 1,024.





Implementation of Page Table - TLB

- The TLB contains only a few of the page-table entries.
- When a logical address is generated by the CPU, its page number is presented to the TLB.
- If the page number is found, its frame number is immediately available and is used to access memory.
 - take less than 10 percent longer than it would if an unmapped memory reference were used.
- If the page number is not in the TLB (known as a **TLB miss**), a memory reference to the page table must be made.
- When the frame number is obtained, we can use it to access memory
- In addition, we add the page number and frame number to the TLB, so that they will be found quickly on the next reference.
- If the TLB is already full of entries, the operating system must select one for replacement.
- Replacement policies range from least recently used (LRU) to random.





Implementation of Page Table –TLB (Cont.)

- ❑ Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
 - ❑ Otherwise need to flush at every context switch
- ❑ TLBs typically small (64 to 1,024 entries)
- ❑ On a TLB miss, value is loaded into the TLB for faster access next time
 - ❑ Replacement policies must be considered
 - ❑ Some entries can be **wired down** for permanent fast access





Associative Memory

- Associative memory – parallel search

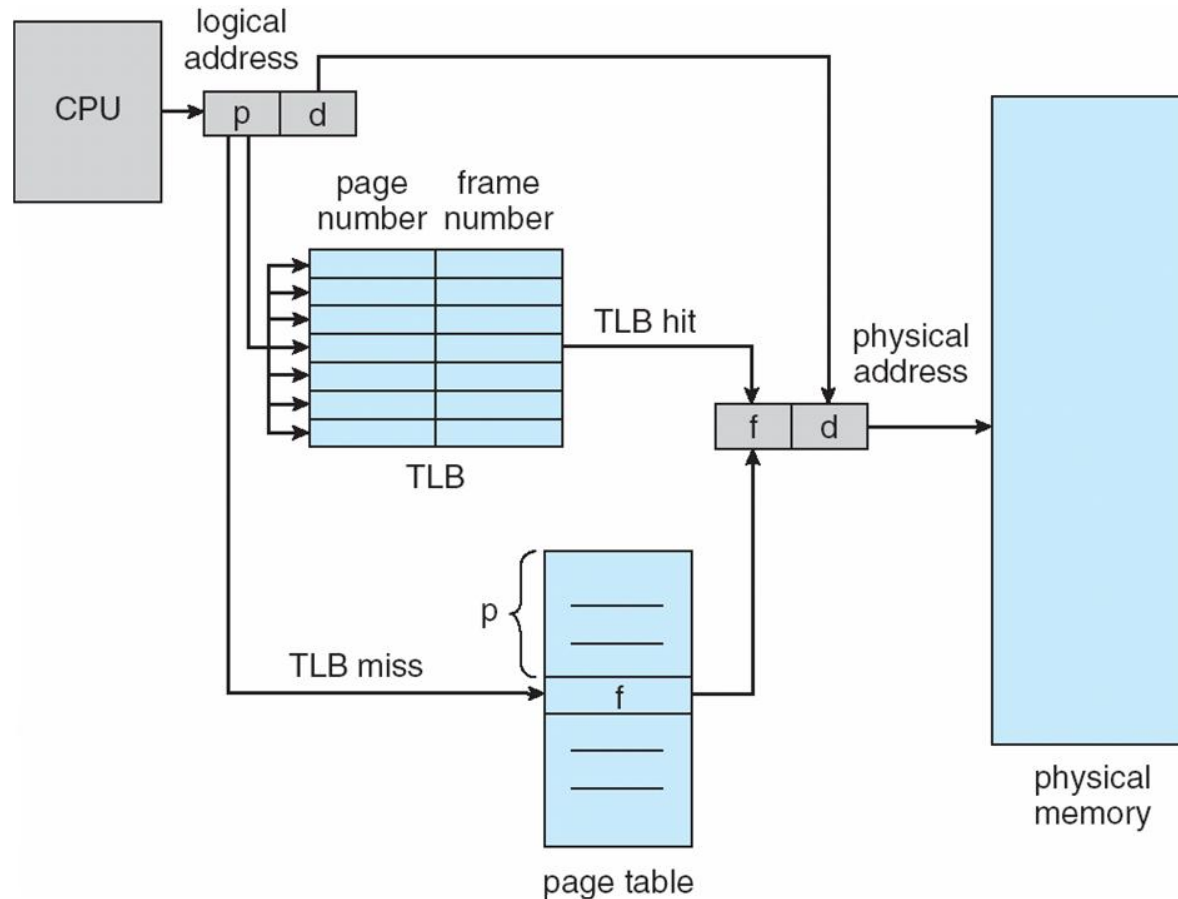
Page #	Frame #

- Address translation (p, d)
 - If p is in associative register, get frame # out
 - Otherwise get frame # from page table in memory





Paging Hardware With TLB





Effective Access Time

- Associative Lookup = ε time unit
 - Can be $< 10\%$ of memory access time
- Hit ratio = α
 - Hit ratio – percentage of times that a page number is found in the TLB
- Consider $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ for TLB search, 100ns for memory access
- **Effective Access Time (EAT)**

$$\text{EAT} = 0.80 \times 120 + 0.20 \times 200 = 140\text{ns}$$
- suffer a 40-percent slowdown in memory-access time (from 100 to 140 nanoseconds).
- Consider more realistic hit ratio $\rightarrow \alpha = 99\%$, $\varepsilon = 20\text{ns}$ for TLB search, 100ns for memory access
 - $\text{EAT} = 0.98 \times 120 + 0.02 \times 200 = 122\text{ns}$
- This increased hit rate produces only a 22-percent slowdown in access time





Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
 - these bits are kept in the page table
 - Every reference to memory goes through the page table to find the correct frame number.
 - At the same time that the physical address is being computed, the protection bits can be checked to verify that no writes are being made to a read-only page.
 - An attempt to write to a read-only page causes a hardware trap to the operating
 - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use **page-table length register (PTLR)**
- Any violations result in a trap to the kernel





Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page <i>n</i>

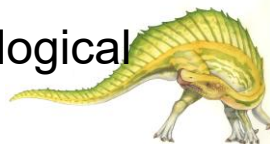




Shared Pages

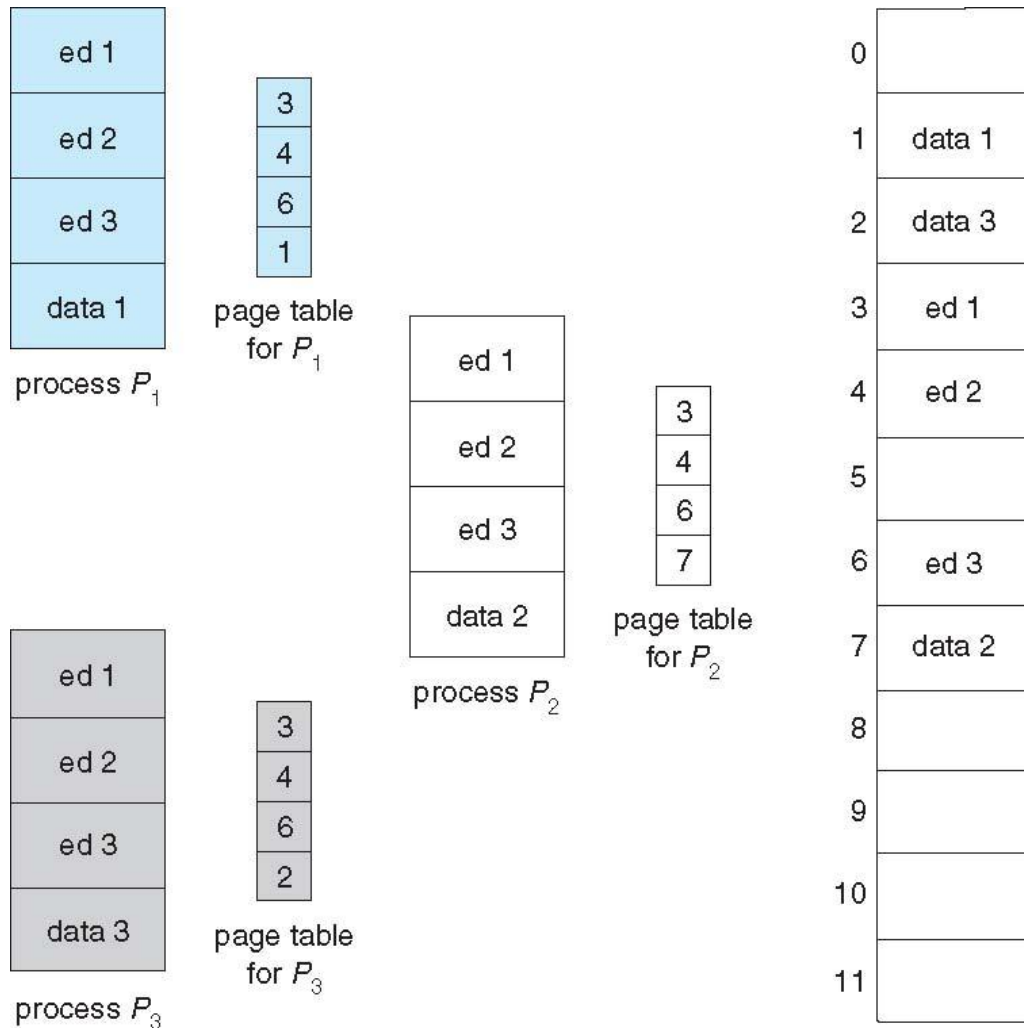
□ Shared code

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed
- Reentrant code is non-self-modifying code: it never changes during execution.
 - Thus, two or more processes can execute the same code at the same time.
- Each process has its own copy of registers and data storage to hold the data for the process's execution.
- The data for two different processes will, of course, be different.
- Only one copy of the editor need be kept in physical memory.
- Each user's page table maps onto the same physical copy of the editor, but data pages are mapped onto different frames
- The pages for the private code and data can appear anywhere in the logical address space





Shared Pages Example





Structure of the Page Table

- ❑ Memory structures for paging can get huge using straight-forward methods
 - ❑ Consider a 32-bit logical address space as on modern computers
 - ❑ Page size of 4 KB (2^{12})
 - ❑ Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - ❑ If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone
 - ▶ That amount of memory used to cost a lot
 - ▶ Don't want to allocate that contiguously in main memory
- ❑ Hierarchical Paging
- ❑ Hashed Page Tables
- ❑ Inverted Page Tables





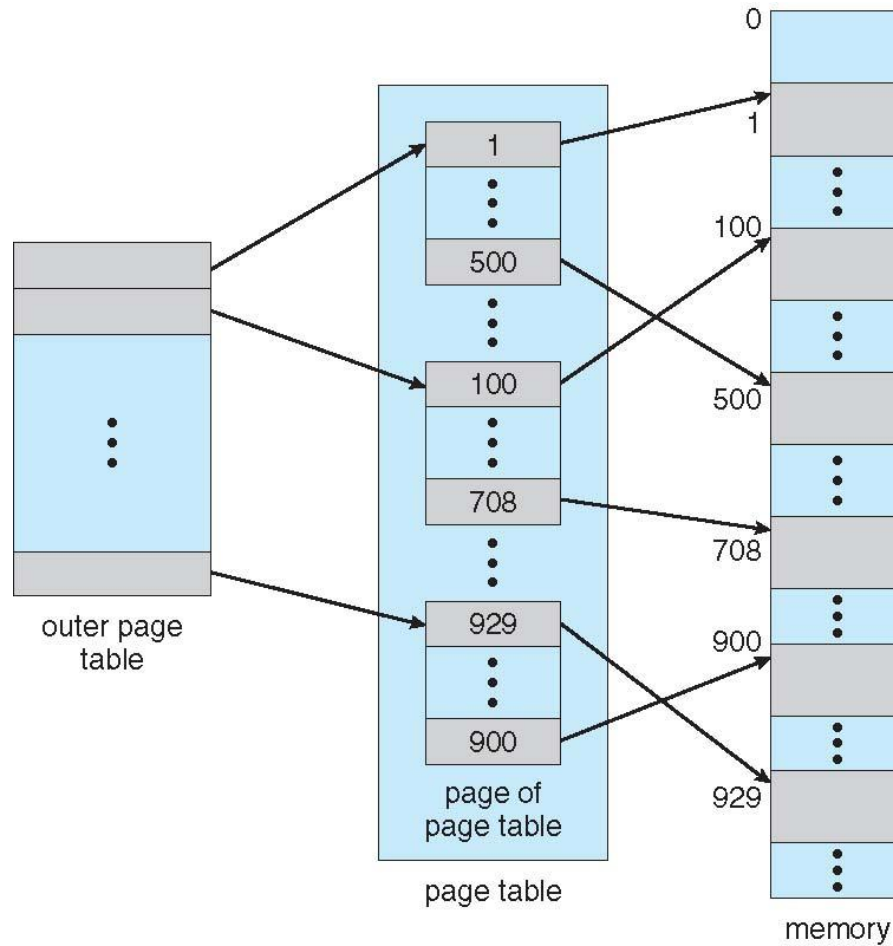
Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table





Two-Level Page-Table Scheme





Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
 - a page number consisting of 22 bits
 - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
 - a 12-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:

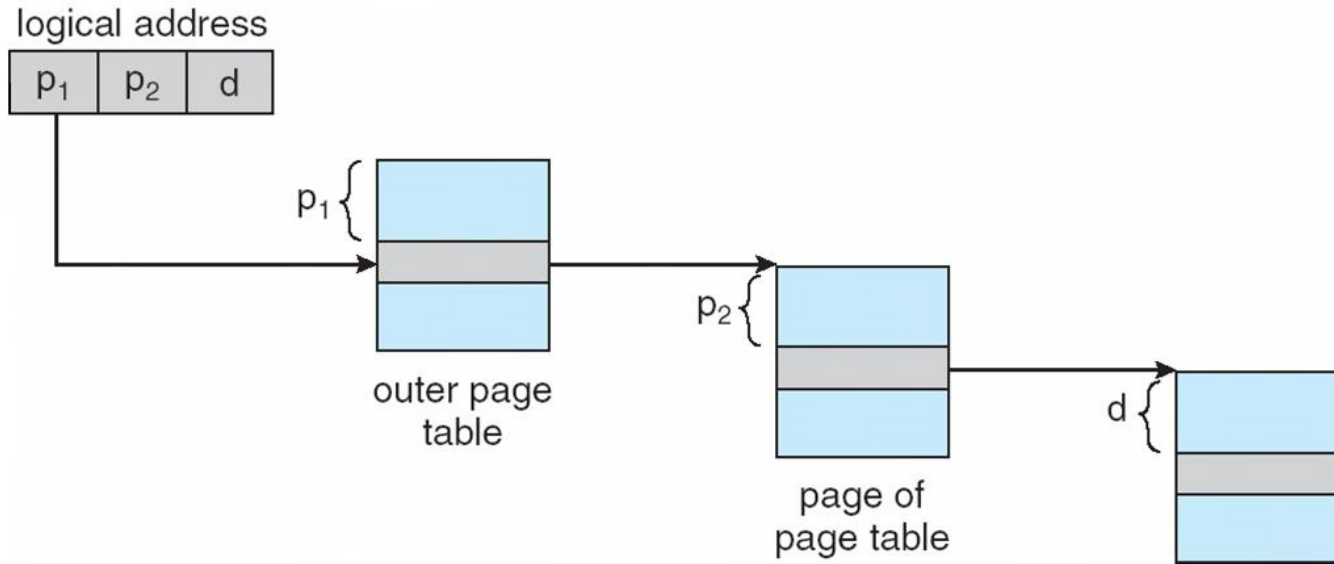
page number		page offset
p_1	p_2	d
12	10	10

- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**





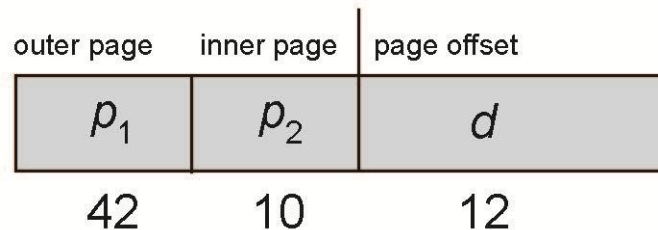
Address-Translation Scheme





64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2^{nd} outer page table
- But in the following example the 2^{nd} outer page table is still 2^{34} bytes in size
 - ▶ And possibly 4 memory access to get to one physical memory location





Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12





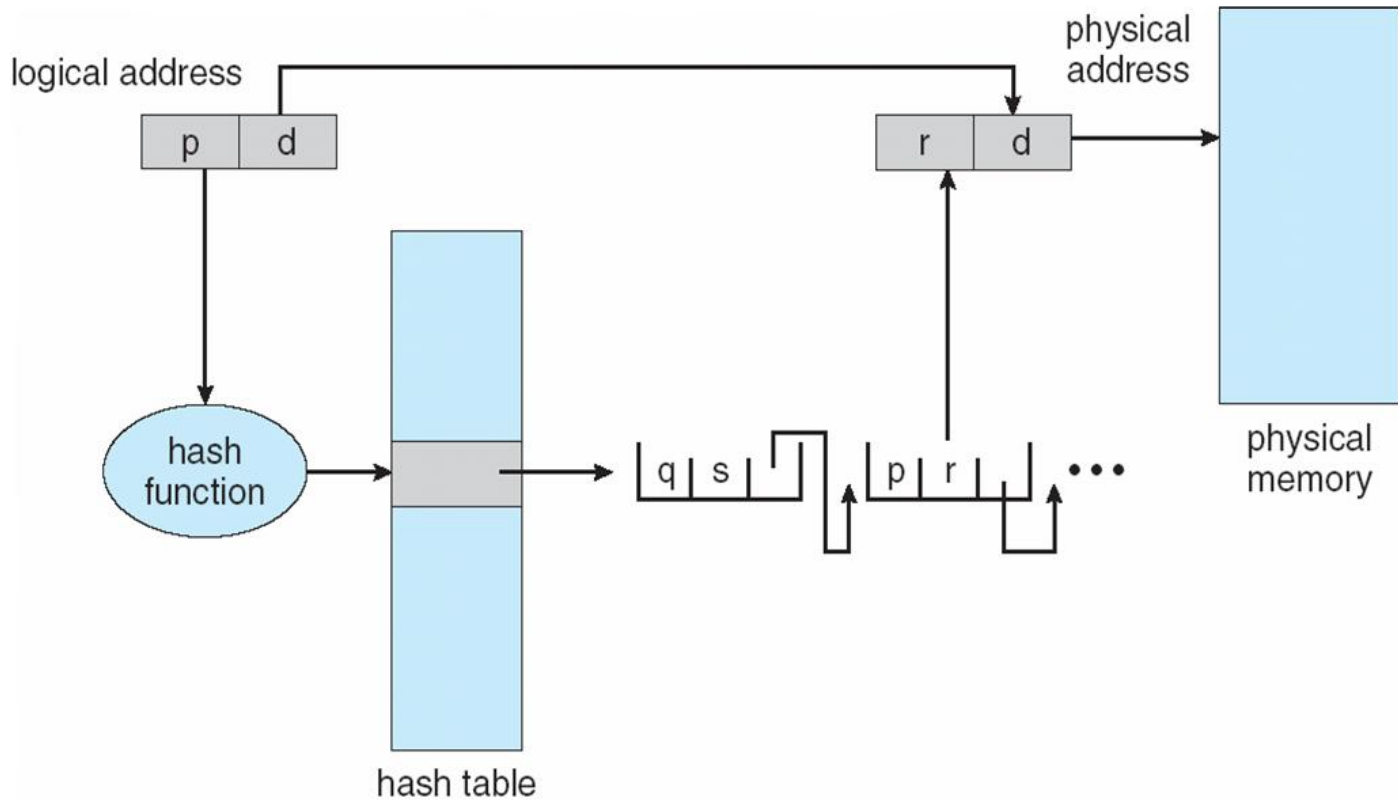
Hashed Page Tables

- ❑ Common in address spaces > 32 bits
- ❑ The virtual page number is hashed into a page table
 - ❑ This page table contains a chain of elements hashing to the same location
- ❑ Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- ❑ Virtual page numbers are compared in this chain searching for a match
 - ❑ If a match is found, the corresponding physical frame is extracted
- ❑ Variation for 64-bit addresses is **clustered page tables**
 - ❑ Similar to hashed but each entry refers to several pages (such as 16) rather than 1
 - ❑ Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)





Hashed Page Table





Inverted Page Table

- ❑ Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- ❑ One entry for each real page of memory
- ❑ Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- ❑ Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- ❑ Inverted page tables often require that an address-space identifier) be stored in each entry of the page table, since the table usually contains several different address spaces mapping physical memory.
- ❑ Storing the address-space identifier ensures that a logical page for a particular process is mapped to the corresponding physical page frame





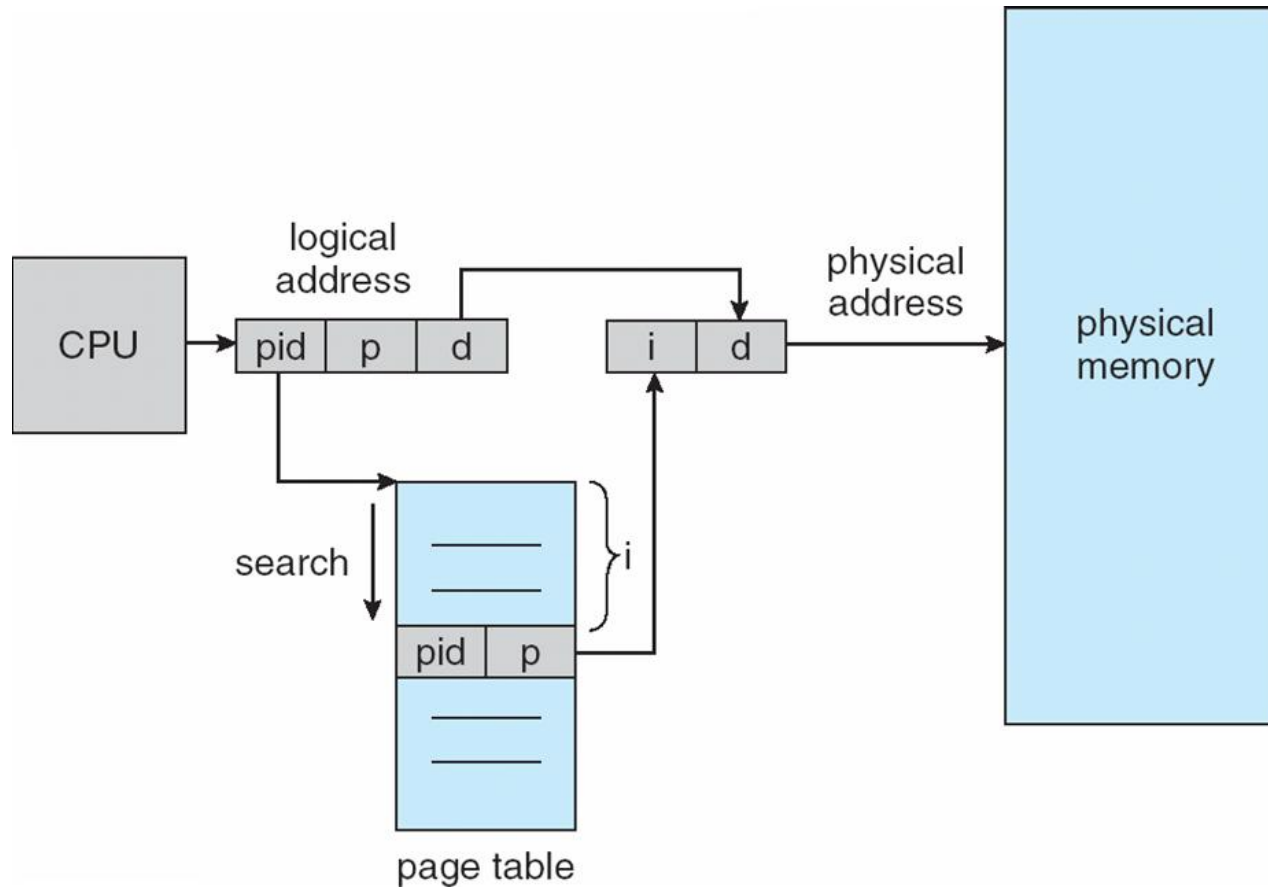
Inverted Page Table Contd...

- Each virtual address in the system consists of a triple:
 <process-id, page-number, offset>.
 process-id assumes the role of the address-space identifier
- When a memory reference occurs, part of the virtual address, consisting of <process-id, pagenumber>, is presented to the memory subsystem.
- The inverted page table is then searched for a match. If a match is found—say, at entry i —then the physical address < i , offset> is generated.
- If no match is found, then an illegal address access has been attempted.
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
 - TLB can accelerate access
- But how to implement shared memory?
 - One mapping of a virtual address to the shared physical address





Inverted Page Table Architecture





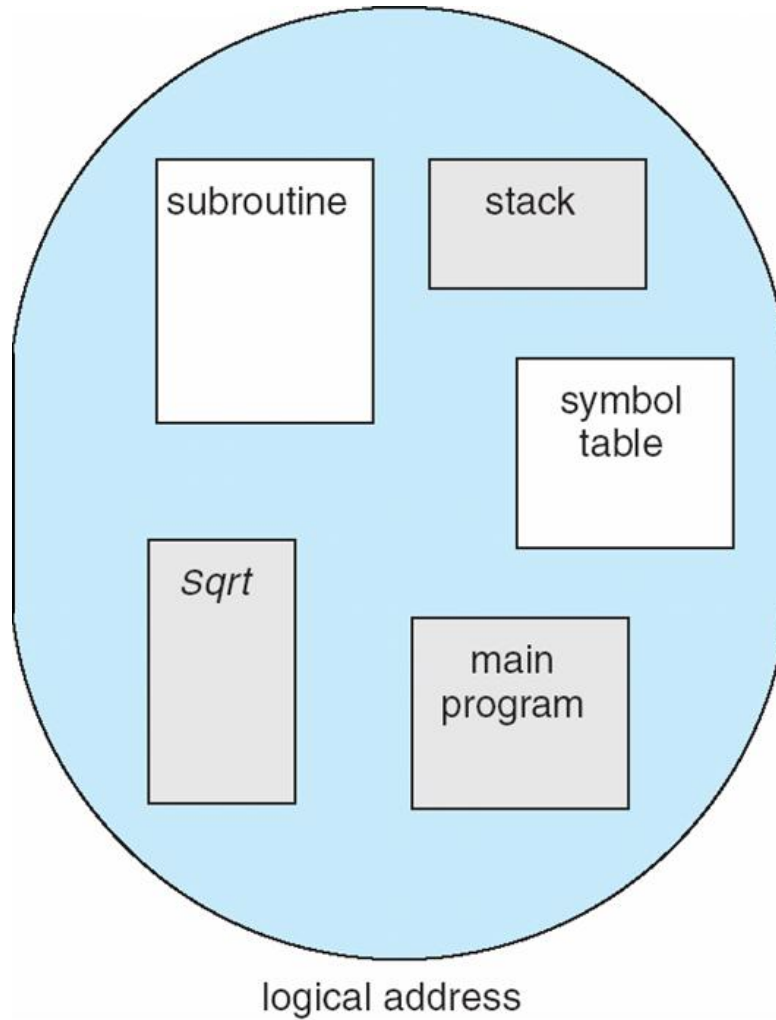
Segmentation

- ❑ Memory-management scheme that supports user view of memory
- ❑ A program is a collection of segments
 - ❑ A segment is a logical unit such as:
 - main program
 - procedure
 - function
 - method
 - object
 - local variables, global variables
 - common block
 - stack
 - symbol table
 - arrays





User's View of a Program





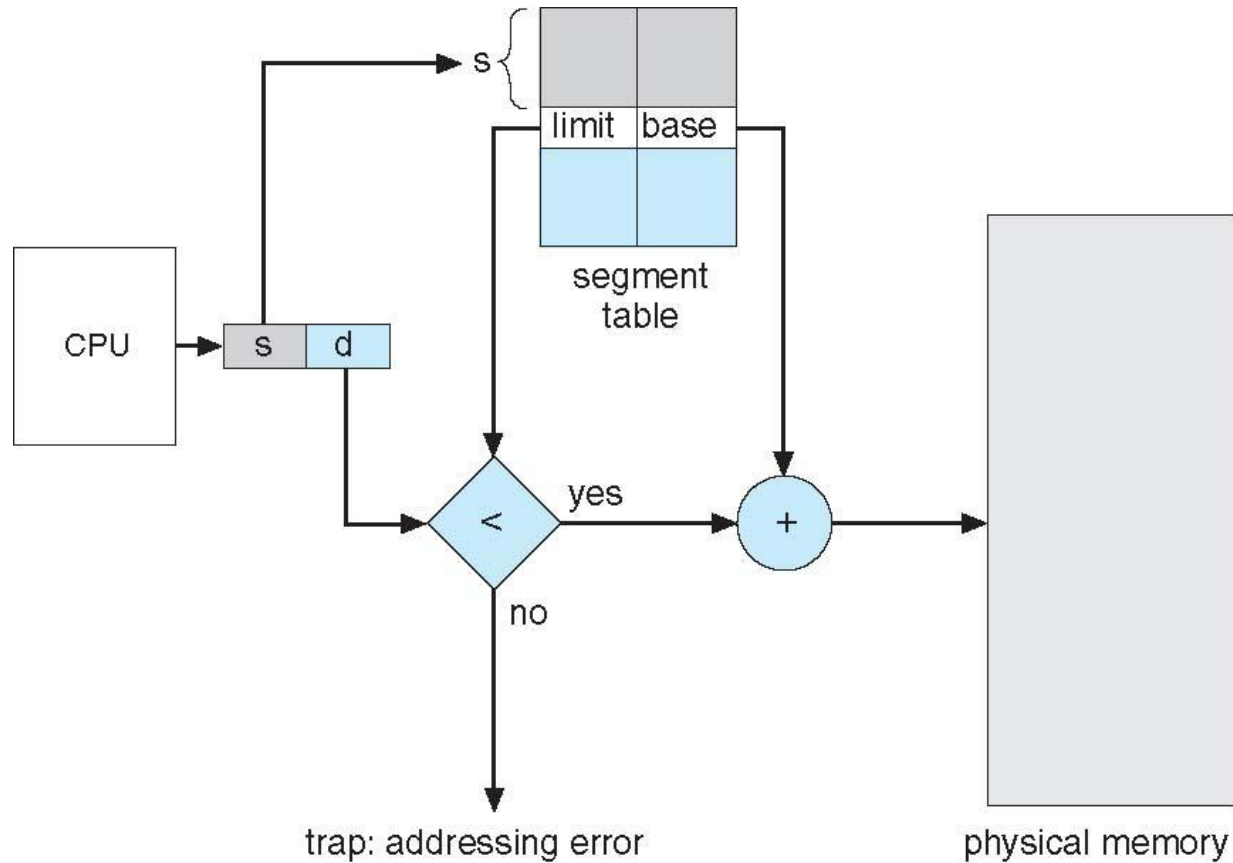
Segmentation Hardware

- Logical address consists of a two tuple:
 <segment-number, offset>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - **Segment base** – contains the starting physical address where the segments reside in memory
 - **Segment limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
 segment number **s** is legal if **s** < **STLR**



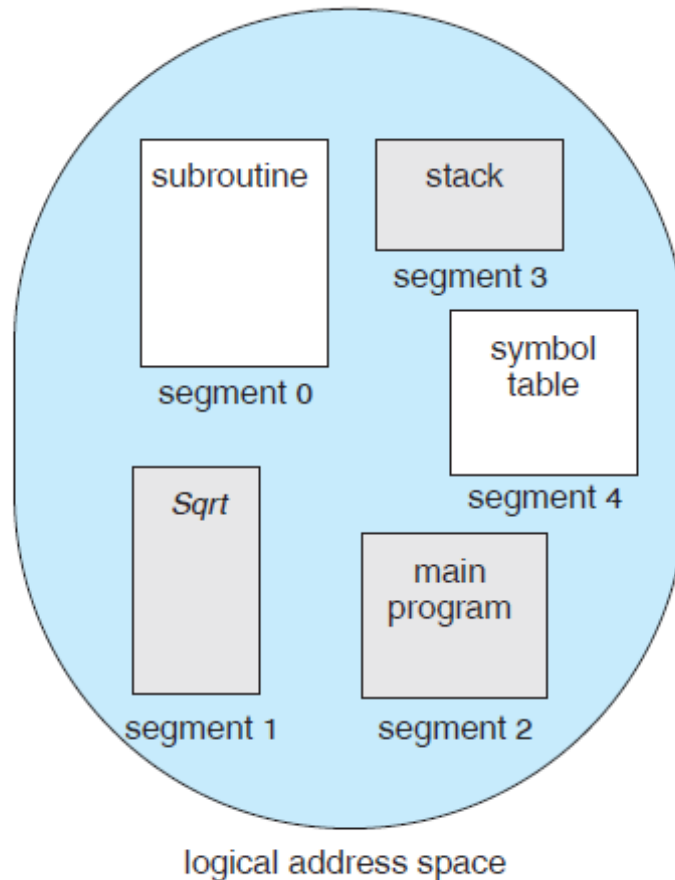


Segmentation Hardware



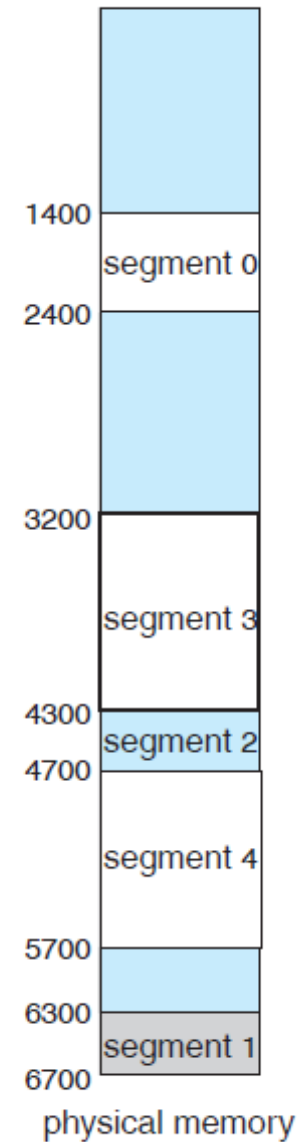


Example of Segmentation



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table





Acknowledgement

- This slides are borrowed from the ppt of Operating Systems Concepts 9Th Edition By Silberschatz, Galvin and Gagne with some modifications done





Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.



End of Chapter 8

